Appendices to Chapter 5:
Detailed Runtime Workflows and Learning Mechanisms

Thesis Documentation

February 8, 2026

Contents

1 Appendix A: Detailed Runtime Monitoring and Decision Workflows

This appendix provides detailed implementation-level workflows for runtime monitoring, state construction, and adaptation option selection processes described at the architectural level in Chapter 5.

1.1 A.1 Monitoring and State Construction Workflow

This section details the step-by-step monitoring and state construction process, including preprocessing, feature extraction, and state validation mechanisms.

1.1.1 Data Preprocessing Pipeline

The data preprocessing phase transforms raw sensor data into preprocessed data suitable for feature extraction. The pipeline includes:

1. **Filtering**: Remove noise and outliers using statistical filters
2. **Normalization**: Scale values to common ranges (e.g., [0, 1] or [-1, 1])
3. **Missing-value handling**: Apply imputation strategies for incomplete sensor readings

1.1.2 Feature Extraction Process

The **Feature Extractor** component processes preprocessed data to generate a compact state representation, deriving goal-relevant attributes while reducing dimensionality. The extracted features are organized into categories:

- **Performance metrics**: Latency, throughput, response time
- **Resource utilization**: CPU usage, memory consumption, network bandwidth
- **Environmental conditions**: Workload intensity, request arrival patterns

1.1.3 State Vector Interface Contract

The state vector $s_t \in \mathbb{R}^d$ serves as the contract between monitoring, learning, and planning. The contract is formally defined as:

- **Structure**: $s_t \in \mathbb{R}^d$ where d is the dimensionality determined by goal-relevant feature categories
- **Feature categories**: Performance metrics, resource utilization, environmental conditions
- **Stability**: Feature values normalized to common range, ensuring consistent scale
- **Contract properties**: All components (Monitor, RS-DRL, Analyzer) operate on same representation

1.1.4 State Validation and Dissemination

The state vector is validated to ensure all feature values are within expected bounds. Once validated, the state is disseminated:

- To Analyzer (Stage 4) for goal evaluation
- To RS-DRL module (Stage 2) for learning or inference
- To Knowledge repository for historical reference

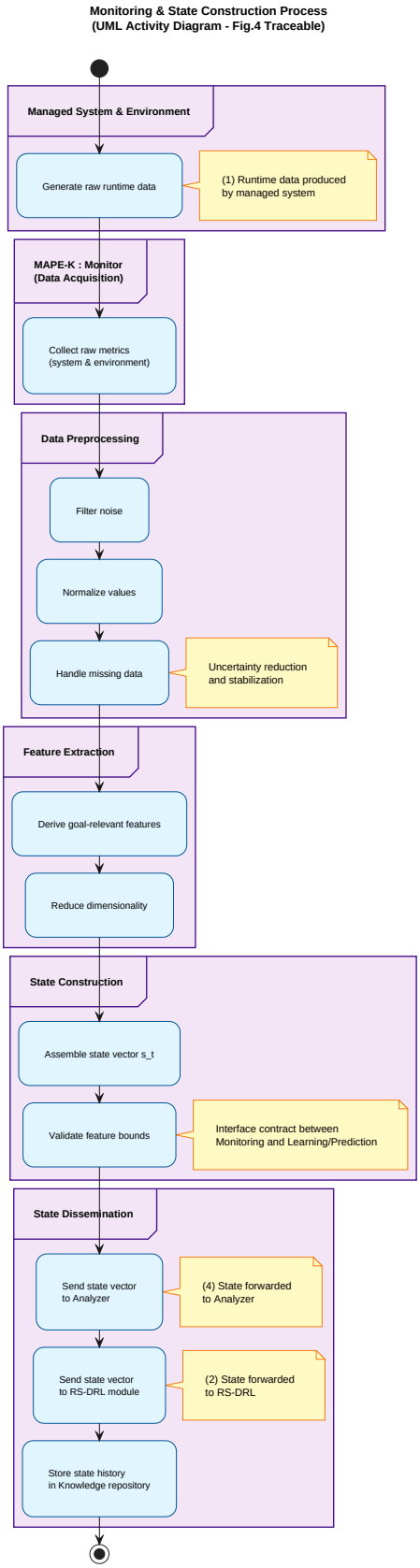

Figure 1: Detailed monitoring and state construction workflow. Raw runtime data is filtered, abstracted, and transformed into a validated state vector suitable for both RS-DRL learning and runtime adaptation analysis.

1.2 A.2 Runtime Adaptation Option Selection Workflow

This section details the runtime adaptation option selection process, including model retrieval, Q-value evaluation, and selection mechanics.

1.2.1 Planning Process Initiation

The planning process is triggered when the Analyzer detects goal violations or risks (Stage 5). Upon this detection, the Analyzer notifies the Adaptation Option Predictor embedded in the Planner.

1.2.2 Trained Model Retrieval

Upon receiving the prediction trigger, the Adaptation Option Predictor retrieves the trained RS-DRL model from the Knowledge repository (Stage 6). The retrieval process accesses the most recently updated model stored during asynchronous learning episodes (Stage 3).

1.2.3 Q-Value Evaluation

The predictor receives the current state vector from the monitoring process and evaluates Q-values for all available adaptation options using the trained deep Q-network (DQN). The DQN estimates Q-values for each action given the current system state:

$$Q(s_t, a; \theta) \rightarrow \text{estimated long-term value of action } a \text{ in state } s_t$$

where θ represents the neural network parameters learned during training.

1.2.4 Option Selection Strategy

The Adaptation Option Predictor selects the best adaptation option based on evaluated Q-values. At runtime, the selection strategy is typically purely exploitative:

$$a^* = \arg\max_{a \in \mathcal{A}} Q(s_t, a; \theta)$$

This exploitation strategy ($\varepsilon \approx 0$) ensures that the system leverages learned knowledge to make optimal decisions during runtime adaptation, avoiding exploration that could lead to suboptimal choices.

During training, an ε-greedy policy is used:

- With probability ε: select random action (exploration)
- With probability $1 - \varepsilon$: select action with highest Q-value (exploitation)

However, at runtime, $\varepsilon \approx 0$, making the policy purely exploitative.

1.2.5 Process Completion

The predicted adaptation option (Stage 7) completes the planning phase. This option is forwarded for verification (Appendix B) and subsequent execution.

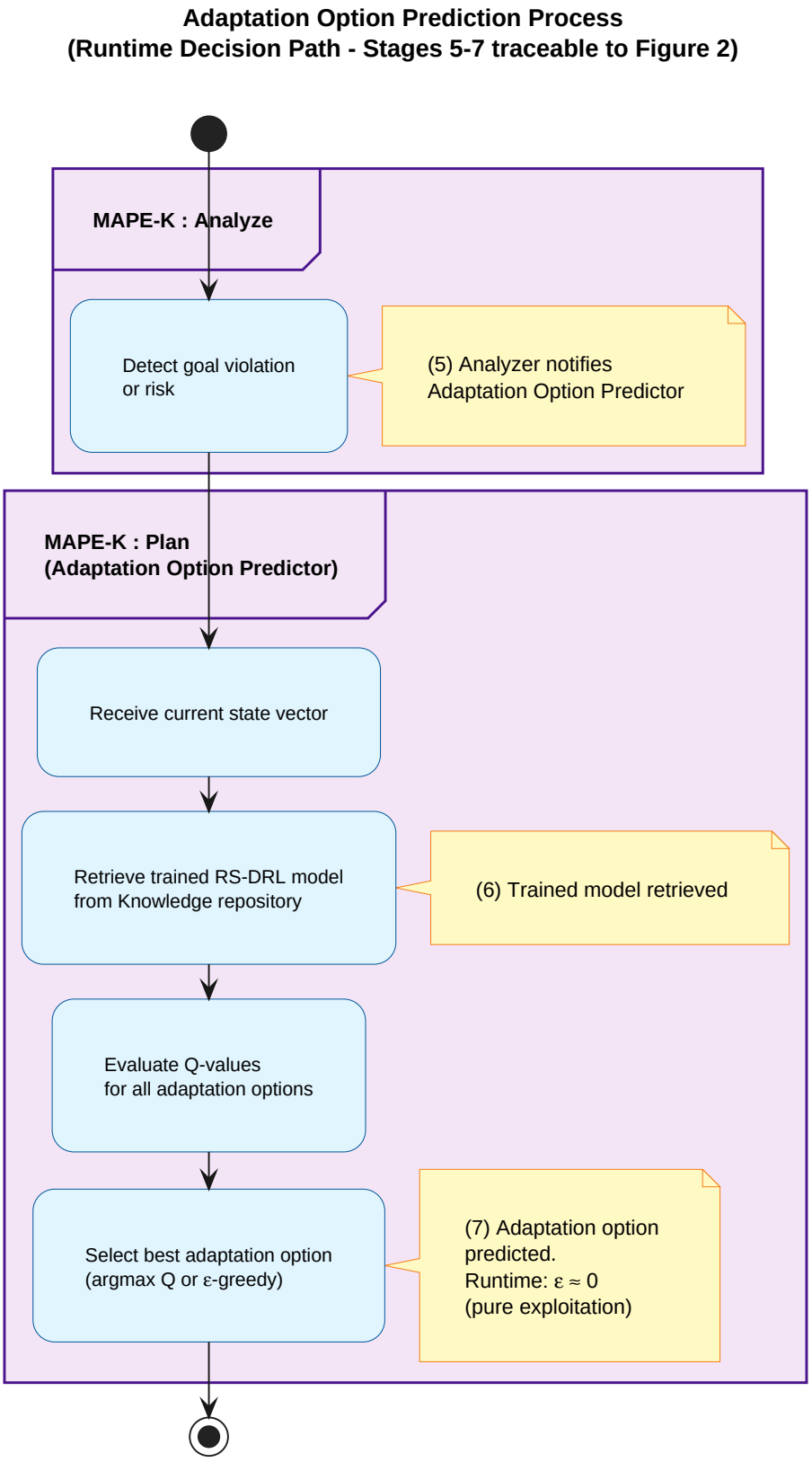

Figure 2: Runtime adaptation option selection workflow. Upon detection of goal violations, the trained RS-DRL model is retrieved and used to predict the most suitable adaptation option through Q-value evaluation.

2 Appendix B: Adaptation Option Verification under Uncertainty

This appendix provides detailed implementation of the adaptation option verification process using runtime models and statistical model checking.

2.1 B.1 Runtime Model Management

The verification process employs runtime models that reflect the current system architecture and context. Runtime models are abstractions of system behavior that capture:

- System components and their interactions
- Environmental conditions and uncertainties
- Quality-of-service attributes
- Adaptation effects on system behavior

2.1.1 Model Parameterization

When a candidate adaptation option is selected by the Adaptation Option Predictor, the verifier retrieves a runtime model and parameterizes it with:

- Current system state s_t
- Candidate adaptation option a
- Environmental uncertainty parameters

This parameterization creates a specific model instance that can be analyzed to predict the adaptation's effects under uncertainty.

2.2 B.2 Statistical Model Checking with UPPAAL-SMC

The verification process employs statistical model checking, implemented via UPPAAL-SMC, to estimate the probability of satisfying adaptation goals under uncertainty.

2.2.1 UPPAAL-SMC Invocation

The UPPAAL-SMC engine executes probabilistic simulations, exploring different execution paths and environmental conditions. The statistical approach handles uncertainty by considering multiple possible futures rather than relying on a single deterministic prediction.

2.2.2 Simulation Configuration

The UPPAAL-SMC verification process operates under the following configuration:

- **Number of simulation runs**: Chosen to achieve desired statistical confidence
- **Time horizon**: Set according to adaptation time scales
- **Uncertainty model**: Probabilistic distributions for environmental parameters
- **Goal properties**: Specified as temporal logic formulas

2.2.3 Assumptions and Limitations

The UPPAAL-SMC verification process operates under these assumptions:

1. Runtime model abstracts system behavior with sufficient fidelity to capture relevant adaptation effects
2. Simulation bounds (number of runs, time horizons) balance statistical confidence with computational cost
3. Estimated probabilities provide statistical confidence intervals rather than absolute guarantees
4. Model abstraction fidelity determines the reliability of verification results

2.3 B.3 Quality Evaluation and Decision Justification

The verification process extracts quality-of-service (QoS) metrics from simulation results, estimating quality attributes such as energy consumption, packet loss, and latency.

2.3.1 QoS Metric Extraction

From the UPPAAL-SMC simulation traces, the verifier extracts:

- **Energy consumption**: Estimated power usage under the candidate adaptation
- **Packet loss rate**: Probability of communication failures
- **Response latency**: Expected service response times
- **Throughput**: Data processing capacity

2.3.2 Decision Justification Output

During planning (runtime), verification provides **decision justification**:

- Quality estimates for the predicted adaptation option
- Goal satisfaction probabilities under uncertainty
- Confidence intervals for quality attributes

These quality estimates justify adaptation decisions, providing trustworthiness and explainability for learning-based choices.

2.3.3 Learning Reward Generation

During learning (asynchronous), the same verification process generates **learning rewards**:

- QoS metrics are aggregated as reward values
- Rewards guide the learning process by indicating desirable outcomes
- Reward signals are consumed by the RS-DRL learning process (Appendix C)

Importantly, reward generation is a **by-product** of verification—the primary purpose of verification during runtime is decision justification, not reward computation.

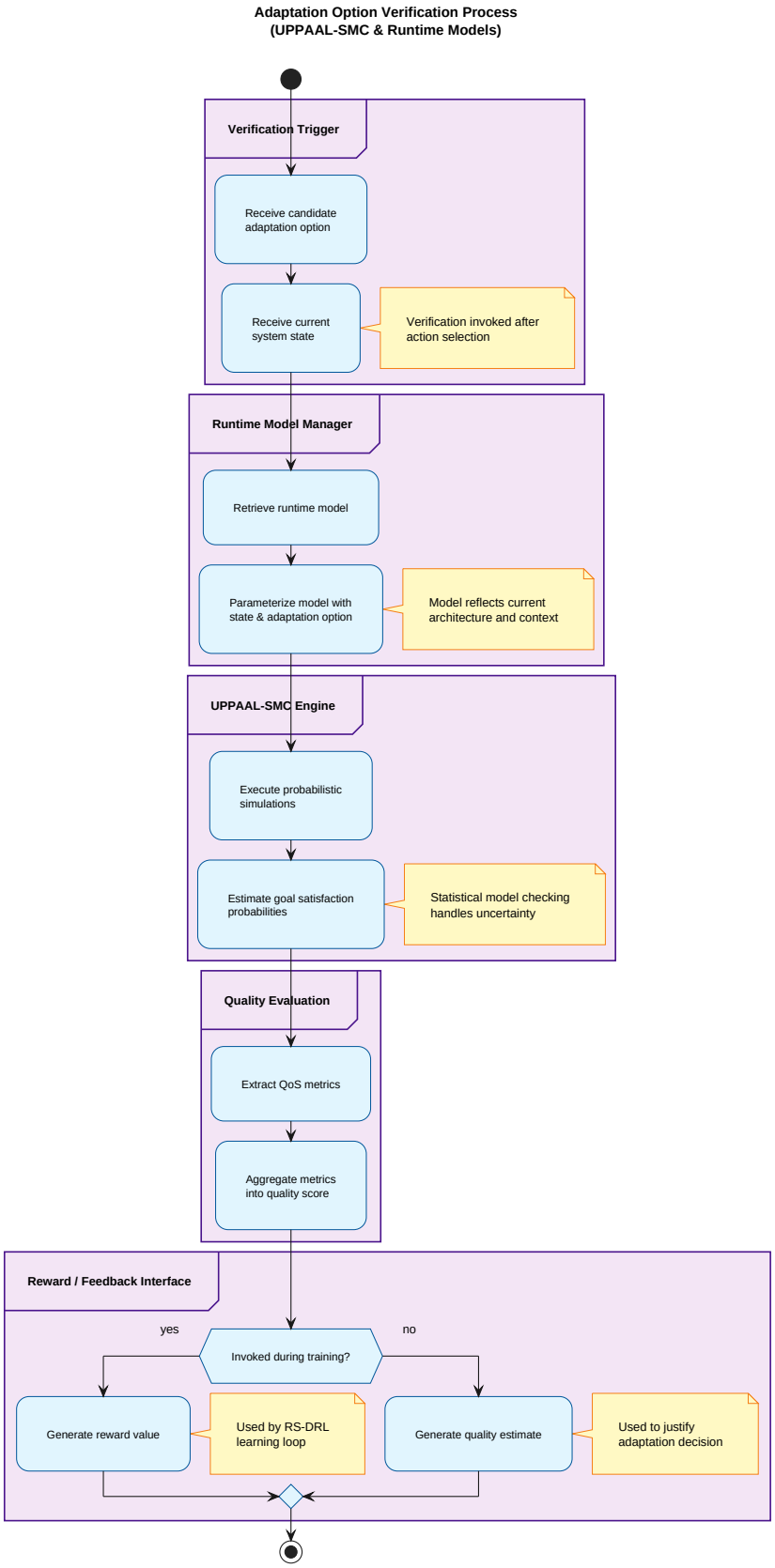

Figure 3: Adaptation option verification workflow using statistical model checking. Runtime models and UPPAAL-SMC are used to estimate goal satisfaction probabilities and extract QoS metrics.

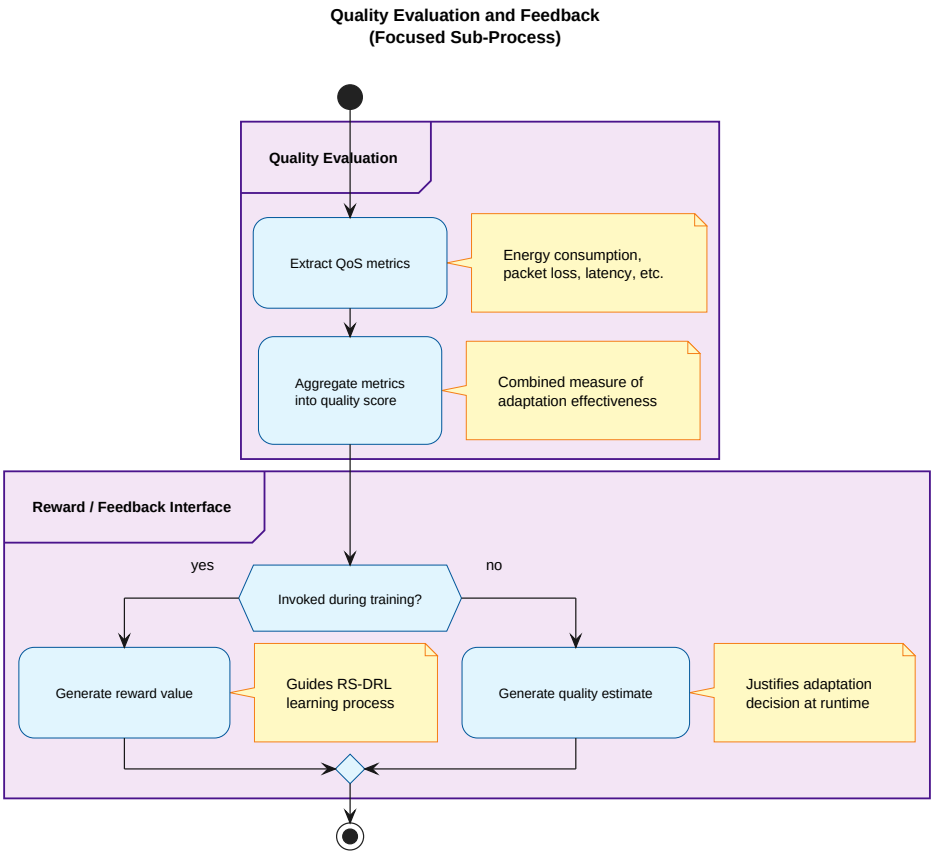

Figure 4: Quality evaluation and feedback generation workflow. QoS metrics are extracted from verification results and transformed into decision feedback (runtime) and learning rewards (asynchronous).

3 Appendix C: RS-DRL Learning and Reward-Shaping Mechanisms

This appendix provides detailed implementation of the RS-DRL learning process, including reward calculation, replay memory, reward reshaping, and policy update mechanisms.

3.1 C.1 Reward Calculation and Feedback Generation

During learning, the learning process consumes verification-derived reward signals generated during the verification phase (Appendix B.3).

3.1.1 Reward Calculator

The **Reward Calculator** assigns rewards based on the verifier's estimates, guiding the learning process of the RS-DRL agent. These verification-derived reward signals are used to update the Q-network, enabling the agent to learn which adaptation actions lead to desirable outcomes.

3.1.2 Verification-to-Reward Interface

The verification process extracts QoS metrics from simulation results. These metrics are aggregated as reward values during learning:

$$r_t = f(\text{energy}, \text{packet\_loss}, \text{latency}, \text{throughput})$$

where f is an aggregation function that combines multiple quality attributes into a scalar reward signal.

By mediating between architectural models and learning, verification ensures that rewards remain aligned with system-level objectives, even as the learning process adapts to new experiences.

3.2 C.2 Replay Memory and Reward Reshaping Strategy

3.2.1 Experience Collection

Training proceeds in episodes, with each episode representing a complete interaction cycle between the RS-DRL agent and the environment. At the beginning of each episode:

1. Training process is initialized
2. Current state s_t is received from monitoring process (Stage 2)
3. Candidate adaptation action is selected using ε-greedy policy
4. Action is executed in environment, producing reward r_t and next state s_{t+1}

This interaction generates the experience tuple $\langle s_t, a_t, r_t, s_{t+1} \rangle$ used for learning.

3.2.2 Epsilon-Greedy Policy During Training

The **Adaptation Action Selector** identifies candidate adaptation actions using an epsilon-greedy policy, balancing exploration and exploitation:

$$a_t = \begin{cases} \text{random action} & \text{with probability } \varepsilon \\ \arg\max_a Q(s_t, a; \theta) & \text{with probability } 1 - \varepsilon \end{cases}$$

The exploration rate ε decays over time, ensuring that the agent transitions from exploration to exploitation as it learns.

3.2.3 Replay Memory Buffer

The **Replay Memory** component stores experience tuples $\langle s_t, a_t, r_t, s_{t+1} \rangle$ as they are generated during environment interaction. Unlike online learning approaches, experience replay decouples data generation from learning, enabling more stable and sample-efficient training.

The replay memory maintains a buffer of past experiences, allowing the learning process to sample minibatches from a diverse set of experiences rather than relying solely on recent, potentially correlated experiences.

Learning is triggered only when the replay buffer contains sufficient experiences, ensuring that model updates are based on representative samples.

3.2.4 Novel Reward-Shaping Mechanism

The **Replay Memory** incorporates a novel reward-shaping mechanism. Unlike conventional experience replay, RS-DRL selectively reshapes certain failed experiences and treats them as successful, mimicking aspects of human learning.

This mechanism:

1. Promotes generalization across heterogeneous environments
2. Accelerates learning by transforming past mistakes into informative training samples

Reward Reshaping Process: After sampling a minibatch from the replay buffer, each experience is evaluated to determine whether it represents a failure transition (i.e., $r = 0$). Failed experiences are then probabilistically reinterpreted as optimistic successes according to a reshaping probability p:

$$r'_t = \begin{cases} r_{\text{success}} & \text{with probability } p \text{ if } r_t = 0 \\ r_t & \text{otherwise} \end{cases}$$

Importantly, reward reshaping is applied only to the sampled minibatch and does not alter the contents of the replay buffer. This design preserves the diversity and integrity of accumulated experiences while mitigating the dominance of failure transitions during early learning stages.

3.3 C.3 Learning Loop and Policy Update Process

3.3.1 Model Training

When the replay buffer is sufficiently populated, minibatches are sampled for learning. During **Model Training**, batches of experiences (with reshaped rewards) are used to update the DQN parameters via backpropagation to minimize the loss function.

3.3.2 Temporal Difference Learning

The training process computes temporal difference (TD) targets using the reshaped rewards:

$$y = r'_t + \gamma \max_{a'} Q(s'_{t+1}, a'; \theta^-)$$

where:

- γ is the discount factor
- s'_{t+1} is the next state
- θ^- represents parameters of the target network

The target network provides stable target values for TD learning, preventing the moving target problem that can destabilize learning.

3.3.3 Loss Function and Gradient Descent

The loss between predicted and target Q-values is computed:

$$L(\theta) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}} \left[(y - Q(s, a; \theta))^2 \right]$$

Gradient descent is performed to update the Q-network parameters:

$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$$

where α is the learning rate.

3.3.4 Target Network Updates

The training process includes periodic updates to the target network, which copies the current Q-network parameters to the target network at regular intervals:

$$\theta^- \leftarrow \theta \quad \text{every } N \text{ training steps}$$

3.3.5 Exploration Rate Decay

Exploration rate decay gradually shifts the policy from exploration to exploitation:

$$\varepsilon \leftarrow \max(\varepsilon_{\min}, \varepsilon \cdot \text{decay\_rate})$$

This ensures that the agent transitions from learning to applying learned knowledge as training progresses.

3.3.6 Model Storage

The trained model is stored in the **Trained Models Repository**, part of the Knowledge repository (Stage 3). This storage makes the updated model available for use by the Adaptation Option Predictor during runtime adaptation episodes.

The repository management maintains versioning and access control for trained models, enabling the system to track model evolution over time and select appropriate models for different contexts.

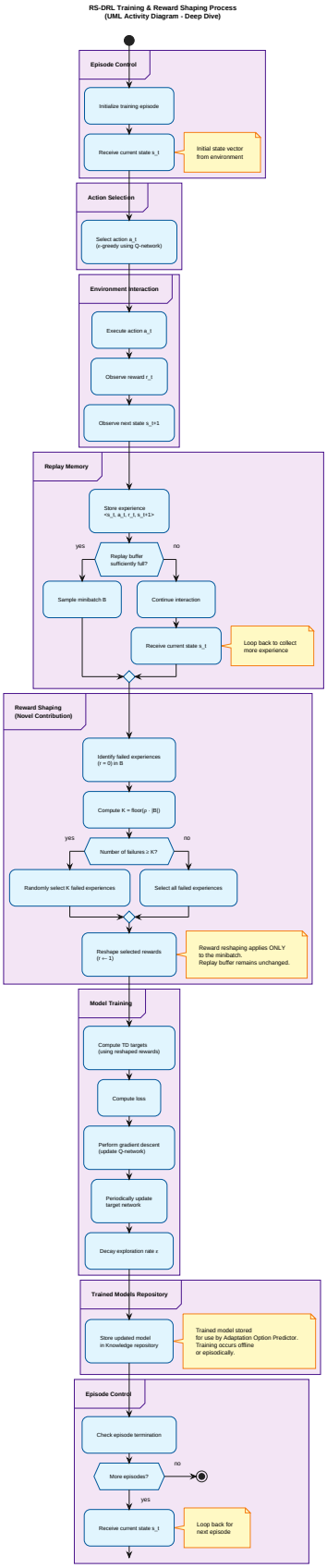

Figure 5: Asynchronous RS-DRL learning and policy update workflow. The figure details episodic interaction with the environment, experience replay, and the novel reward reshaping mechanism applied during minibatch learning.

4 Appendix Summary

These appendices provide implementation-level details supporting the architectural descriptions in Chapter 5:

- **Appendix A** detailed runtime monitoring, state construction, and adaptation option selection workflows
- **Appendix B** explained verification mechanisms using runtime models and statistical model checking
- **Appendix C** described RS-DRL learning internals, including the novel reward-shaping mechanism

These details are algorithmic and statistical in nature, complementing the architectural focus of the main chapter. Readers requiring full technical transparency can consult these appendices, while readers focusing on system-level architecture can rely on the summaries provided in Chapter 5.